

CSE 341 Final Project Proposal

General Info

Student Names in group

Santiago Irigoyen

Proposed Application Name

GameVault API

Application Info

What will the API do?

The GameVault API is a RESTful backend application designed to manage video game information, user accounts, reviews, and personal collections. Users will be able to browse games, create reviews, manage collections of games, and authenticate securely using OAuth.

The API will support full CRUD functionality for all collections and will be connected to MongoDB Atlas.

How will your API utilize a login system?

The API will use Google OAuth 2.0 authentication with Passport.js. Users will authenticate using their Google accounts, allowing secure login and session management.

Authenticated users will be able to:

- Create and edit reviews
- Create and manage collections
- Modify user profile information

Unauthenticated users will only be able to access public game information.

What database will you use?

The application will use MongoDB Atlas as the cloud database solution.

How will the data be stored in your database?

The database will contain the following collections:

Games

Stores video game information.

- title
- genre
- developer
- publisher
- releaseYear
- platform
- rating
- multiplayer
- price

Consoles

Stores gaming console information.

- name
- manufacturer
- releaseYear
- generation
- storageCapacity
- onlineSupport
- price

Users

Stores user account information.

- username
- email
- oauthId
- favoriteGenre
- accountCreated

Reviews

Stores user reviews for games and consoles.

- userId
- itemId
- reviewText

- score
- createdAt

How would a frontend be able to manage authentication state based on the data you provide?

The frontend application will receive authentication session information after successful OAuth login. Protected routes will verify user authentication before allowing actions such as creating reviews or modifying collections.

The API will return authenticated user data, so the frontend can determine login state and manage protected UI features.

What pieces of data in your app will need to be secured? How will you demonstrate web security principles in the development of this app?

The following information will be secured:

- OAuth login information
- User account information
- Session data
- MongoDB credentials
- Environment variables

Security principles demonstrated:

- OAuth authentication
- Environment variable protection is used .env
- Validation using express validator
- Protected API routes
- Error handling
- Secure deployment on Render

What file structure and program architecture will you use for this project (how will you organize your node project)? Why?

The project will follow a modular MVC-inspired architecture.

Structure:

- controllers/
- routes/
- database/
- middleware/
- swagger/
- tests/

- models/
- utilities/

This structure improves scalability, readability, maintainability, and team collaboration.

What are potential stretch challenges that you could implement to go above and beyond?

Potential stretch goals:

- Search and filtering system
- Favorite games system
- Pagination for large data collections
- JWT authentication support
- User profile customization
- Advanced review rating calculations
- Admin role permissions

API Endpoint Planning

For this section, you'll plan out what API endpoints you'll need for your project.

Games

GET /games

GET /games/{id}

POST /games

PUT /games/{id}

DELETE /games/{id}

Consoles

GET /consoles

GET /consoles/{id}

POST /consoles

PUT /consoles/{id}

DELETE /consoles/{id}

Users

GET /users

GET /users/{id}

POST /users

PUT /users/{id}

DELETE /users/{id}

GET /login

GET /logout

Reviews

GET /reviews

GET /reviews/{id}

POST /reviews

PUT /reviews/{id}

DELETE /reviews/{id}

Project Scheduling and Delegation

Plan out what tasks will get completed with each lesson remaining in the semester (Only edit highlighted text).

Week 04 Tasks • Complete Final Project Proposal • Plan database collections • Plan API endpoints • Create project structure • Research OAuth implementation	<i>Project Proposal</i>
Week 05 Tasks • Create GitHub repository • Push project to Render • Create Games CRUD routes • Create Users CRUD routes • Configure MongoDB Atlas • Complete Swagger documentation at /api-docs	• <i>Create Git Repo</i> • <i>Push to Heroku</i> • <i>API DOCUMENTATION is complete and available at route '/api-docs'</i>
Week 06 Tasks • Implement OAuth authentication • Create Reviews CRUD routes • Create Collections CRUD routes • Add validation and middleware • Implement unit testing for GET routes	
Week 07 Tasks • Final testing • Fix deployment issues • Improve API documentation • Record video presentation • Final project submission	

How will you divide up work in your team to ensure the following tasks all get completed?

- HTTP GET, GET (all, single) - Implementation of all GET routes and testing.
- HTTP POST - Implementation of creating routes with validation.
- HTTP PUT - Implementation of updated routes with validation and authentication.
- HTTP DELETE - Implementation of delete routes with protected access.
- Node.js project creation - Setup of Express server, middleware, routing, and architecture.
- Create git repo and share with Group - Management of GitHub repository and version control.

- MongoDB setup - Configuration of MongoDB Atlas database and collections.
- API Swagger documentation for all API routes - Creation and maintenance of Swagger documentation.
- Video presentation of node project, all routes functioning, mongoDB data being modified, and API documentation. - Recording and editing the final presentation video.

Potential Risks and Risk Mitigation Techniques

What are the risks involved with you being able to finish this project in a timely manner?

Potential risks include:

- OAuth configuration difficulties
- Deployment issues with Render
- MongoDB connection problems
- Time management challenges
- Debugging unexpected API errors

How will you mitigate or overcome these risks?

The following strategies will be used:

- Begin implementation early
- Test features incrementally
- Use GitHub commits regularly
- Keep documentation updated
- Use modular architecture for easier debugging
- Review official OAuth and MongoDB documentation
- Maintain backups of working code versions